SYSTEM DESIGN & ARCHITECTURE SPECIFICATION

Neutron Search Engine & Distributed Web Crawler

Version 3.0 — High-Level Design, Low-Level Internals, and Cloud Infrastructure

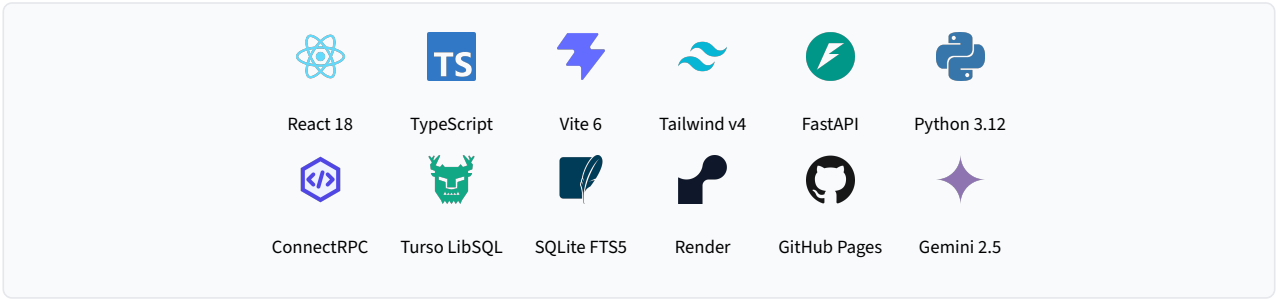
Abstract

Neutron is a privacy-focused web search engine, on-demand crawler, and citation synthesis system structured around three decoupled Python microservices communicating over ConnectRPC/Protobuf, a serverless Turso (LibSQL) FTS5 index, and a static React web client. This specification documents the deployed v3.0 architecture across GitHub Pages and Render Free instances, detailing request routing, hybrid cache-first and server-streaming crawl execution, lexical query normalization and ranking, HTML extraction, bounded deduplication, and storage modeling.

Document Metadata

Author	Md Fahim Sarker
System Release	Neutron v3.0 (`main`)
Runtime	Python 3.12.8 (FastAPI, ConnectRPC) / React 18 (TypeScript, Vite)
Primary Storage	Turso LibSQL (SQLite FTS5 + Zstandard level-3 BLOBs)
Hosting Platforms	GitHub Pages (Frontend) · Render Free, Oregon (3 Web Services)
Source Repository	`https://github.com/fahiiim/Neutron-Web-Crawler`
Public Gateway	`https://neutron-gateway.onrender.com`

Core Technology Stack



Document derived strictly from verified production source files in `services/`, `spaces/crawler/`, `proto/`, `migrations/`, and `frontend/`.

Contents

1 System Overview & Architectural Goals 1

2 Cloud Architecture & Network Topology 2

2.1 Cold-Start & Cross-Origin Resilience 2

3 High-Level Design (HLD): Request Lifecycle & Streaming 3

3.1 Caching Hierarchy, Refresh Merging & Failure Reporting 3

4 Low-Level Design (LLD): Internals & Algorithms 4

4.1 Protobuf Contracts, Streaming Auth & 64-Bit Identifiers 4

4.2 Query Normalization, FTS5 Grouping & Multi-Signal Ranking 4

4.3 Live Discovery, Parsing & Bounded Deduplication 5

5 Storage Schema (`Turso LibSQL FTS5`) 6

6 Latency Budgets, Security Controls & Tooling 7

---

1. System Overview & Architectural Goals

Search engines deployed on low-cost or serverless infrastructure face two competing constraints: full-text database lookups are fast (tens of milliseconds) but limited to previously indexed pages, whereas live multi-source web crawling yields fresh coverage at the cost of network latency, anti-bot blocking, and higher CPU/memory usage.

Neutron resolves this trade-off through an adaptive two-tier retrieval design backed by strict service separation:

- **Edge Separation:** Presentation assets live on GitHub Pages while public API orchestration (`services/gateway`), index query/synthesis (`services/search`), and live web crawling (`services/crawler`) run as isolated Render web services bound by Protobuf v3 contracts (`proto/neutron/v1/neutron.proto`).
- **Cache-First, Stream-Second Execution:** Queries meeting `JIT_CACHE_THRESHOLD` (`5` indexed matches) return immediately from Turso FTS5 (or Gateway's `300 s` `_TTLCache`). Sparse or refreshed queries trigger `CrawlerService.StreamCrawl`, pushing real-time discovery and crawl updates to the browser over Server-Sent Events (SSE).
- **Predictable Free-Tier Resource Bounds:** Shared `0.1 vCPU` / `512 MB` worker limits are respected through `12 KB` SimHash truncation, an `80`-link extraction cap per document, bounded async semaphores (`MAX_CONCURRENT_JIT_FETCHES = 12`), and explicit `8.5 s` batch deadlines.
- **Minimalist Data Retention:** The system sets no cookies, logs no client IP addresses to disk, validates external links via `safeHttpUrl`, and renders reader Markdown strictly through `marked` and `DOMPurify`.

Service / Layer	Primary Entrypoint	Core Responsibility
Static Web Client	`frontend/src/App.tsx`	Search UI, `sessionStorage` cache (5 m), SSE event consumer, Markdown reader modal, and Architecture view.
Edge Gateway	`services/gateway/app.py`	REST/SSE/ConnectRPC coordinator, `_TTLCache`, `slowapi` rate limits, `X-Request-Id` propagation, and cold-start wake pings.
Search Service	`services/search/app.py`	ConnectRPC `SearchService`: wide-pool FTS5 queries, re-ranking, Zstd payload decompression, and Gemini/extractive synthesis.
Crawler Service	`services/crawler/app.py`	ConnectRPC `CrawlerService`: 6 parallel discovery sources, `StreamCrawl`, `robots.txt` cache, `curl_cffi` fetch, and Bloom/SimHash dedup.
Storage Engine	`migrations/03_turso_fts5_schema.sql`	Single Turso LibSQL database storing index metadata, Porter FTS5 triggers, Zstd BLOBs, frontier tasks, and hot topics.

Table 1: Service responsibilities across the Neutron v3.0 codebase.

2. Cloud Architecture & Network Topology

Figure 1 shows the production deployment layout across GitHub, Render (Oregon), Turso, Google Gemini, and external web sources. Because Render Free web services do not expose private mesh networking across separate free instances, `neutron-gateway` communicates with `neutron-search` and `neutron-crawler` over public HTTPS using `pyqwest.HTTPTransport` configured with `certifi` root certificates and shared Bearer authentication (`INTERNAL_API_KEY`).

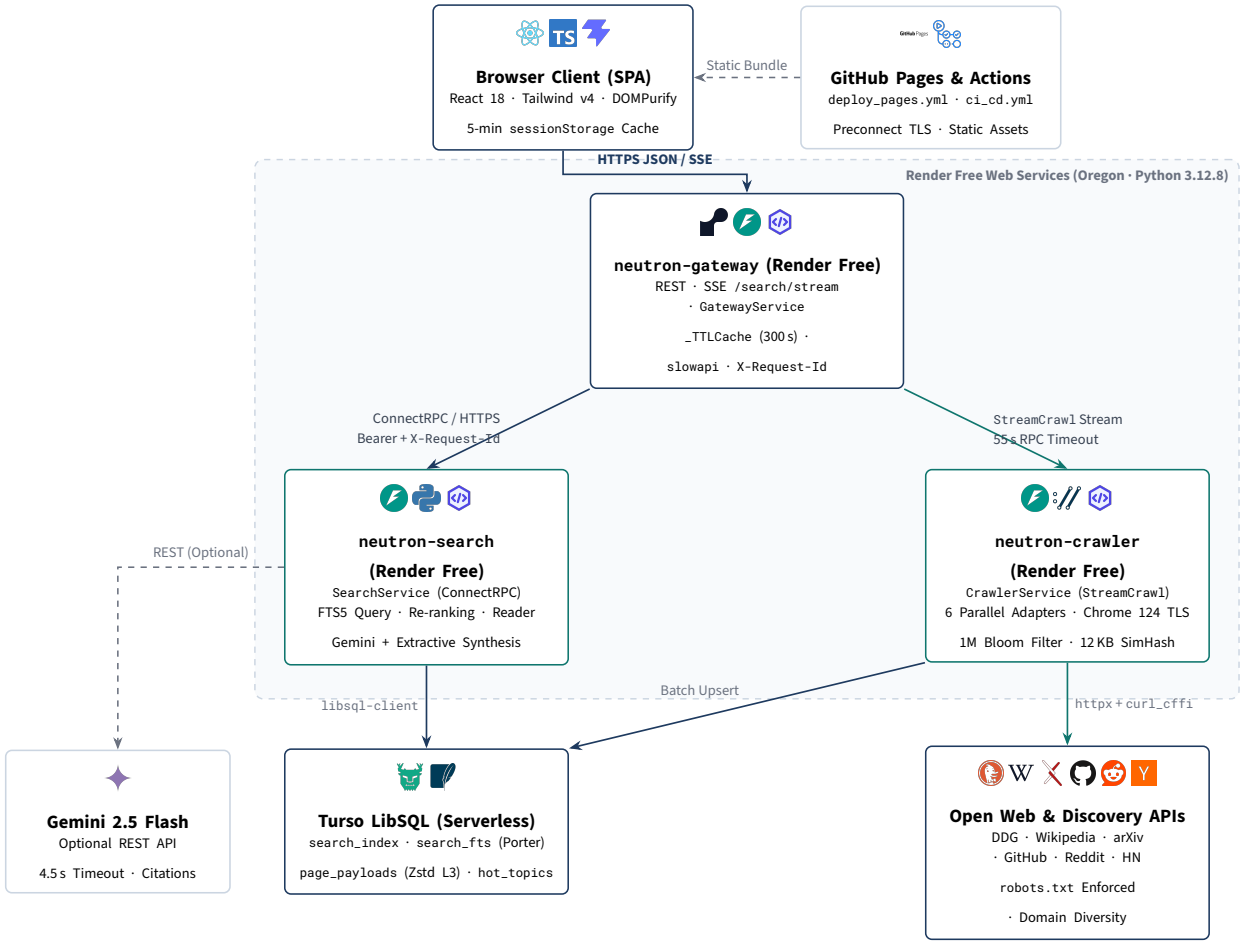


Figure 1: Cloud infrastructure and inter-service communication boundaries.

2.1 Cold-Start & Cross-Origin Resilience

Free-tier Render services sleep after 15 minutes of inactivity and require roughly 30–50 seconds to cold-boot:

- **Auto-Resolved Blueprint URLs:** In `services/common/config.py`, when `RENDER=true` and local overrides are unset, `GatewayServiceSettings` defaults to the public `.onrender.com` hostnames.
- **Throttled Warm-Up Pings:** `_maybe_wake_downstream()` issues background `/health` requests (throttled to at most once every 180 s) to both `neutron-search` and `neutron-crawler` on `/health`, `/stats`, or downstream `502`/`503` failures.
- **Preconnect & Timeout Envelope:** `frontend/index.html` declares `<link rel="preconnect">` to `neutron-gateway.onrender.com`, while `RPC_TIMEOUT_MS=55000` spans cold container boots. Custom `429` and `500` FastAPI exception handlers ensure CORS headers remain attached when downstream calls fail.

3. High-Level Design (HLD): Request Lifecycle & Streaming

Every search query enters either `/search` (JSON), `/api/research` (agent JSON + full Markdown), or `/search/stream` (SSE). Figure 2 depicts the streaming cascade executed for interactive browser searches.

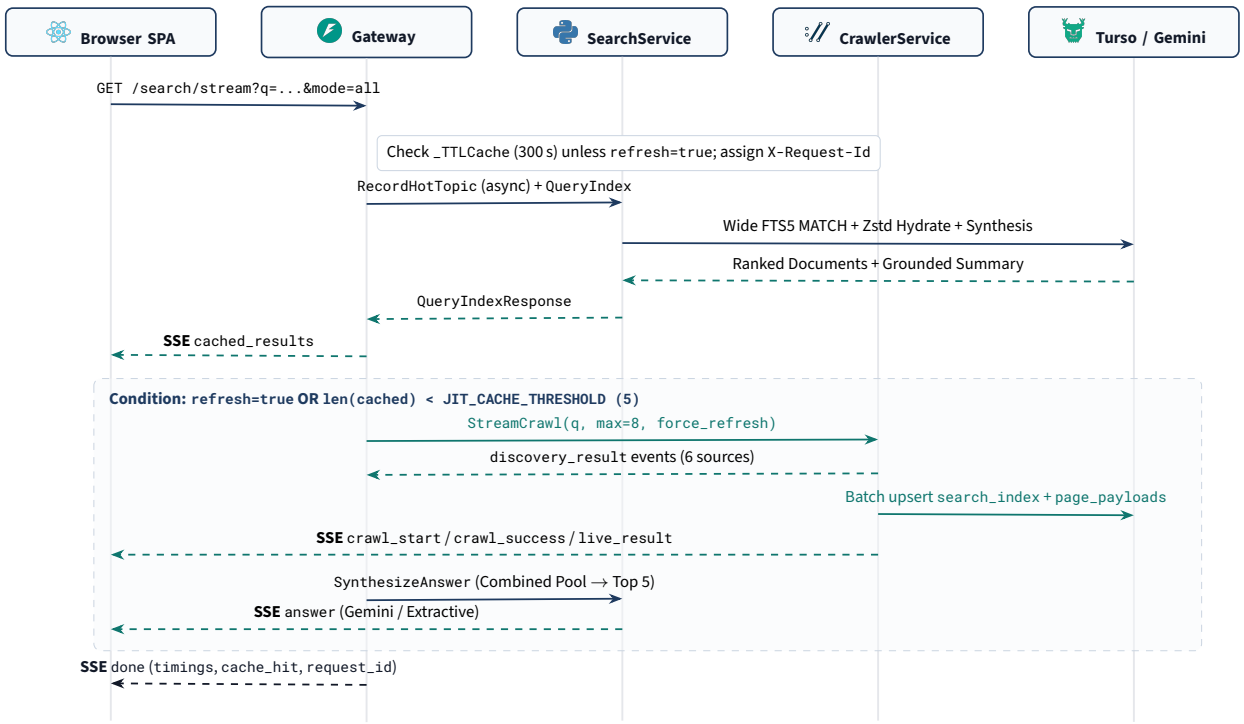


Figure 2: High-Level Design: Cache-first and JIT server-streaming execution sequence.

3.1 Caching Hierarchy, Refresh Merging & Failure Reporting

- **Two-Layer Caching:** The React client checks `sessionStorage` (5-minute TTL keyed by `mode:query`), while `neutron-gateway` checks an in-memory `_TTLCache` (256 query entries at `300` s, plus connected `/stats` for `30` s).
- **In-Place Refresh Replacement:** When `refresh=true` is passed, caches are bypassed and `StreamCrawl` receives empty `skip_urls`. Returned live documents replace stale cached documents at matching URLs inside both `neutron-gateway` and React state, triggering fresh synthesis over up to `max(12, limit)` candidates.
- **Explicit Error Propagation:** If both `SearchService` and `CrawlerService` fail during an SSE stream with zero cached hits, Gateway emits `event: backend_error` so the UI displays a retryable wake-up notice rather than reporting zero matches.

4. Low-Level Design (LLD): Internals & Algorithms

4.1 Protobuf Contracts, Streaming Auth & 64-Bit Identifiers

All service interfaces are defined in `proto/neutron/v1/neutron.proto` and compiled via `scripts/gen_proto.py` (`protoc-gen-py` + `protoc-gen-connectrpc`).

Service	RPC Surface (proto3)	Implementation Details
SearchService	`QueryIndex, SynthesizeAnswer, GetDocument, Suggest, GetTrending, GetStats, RecordHotTopic`	Runs 3x candidate pre-fetch when `mode != "all"` (2x otherwise), re-ranks, decompresses top-5 payloads, and invokes synthesis.
CrawlerService	`DiscoverSeeds, CrawlUrl, DiscoverAndCrawl, StreamCrawl`	Runs concurrent discovery + `curl_cffi` fetch pool; `StreamCrawl` yields `CrawlStreamEvent` messages via `asyncio.Queue` with clean cancellation.
GatewayService	`Search, Suggest, GetStats, GetDocument`	Public ConnectRPC/gRPC-Web mirror mounted alongside REST/SSE endpoints.

Table 2: Compiled ConnectRPC service contracts in `neutron_rpc/neutron/v1/`.

> **Auth Interceptor & JavaScript 64-Bit Precision**
>
> **Unified Unary + Stream Interception:** `InternalAuthInterceptor` (`services/common/rpc.py`) extends `connectrpc.interceptor.MetadataInterceptor[None]`, checking `Authorization: Bearer` or `x-internal-key` with `hmac.compare_digest` across both unary RPCs and `StreamCrawl`.
>
> **64-Bit Hash Serialization:** `hash_url()` maps `xxhash.xxh64` digests to signed 64-bit SQLite integers. Because JavaScript loses integer precision above $2^{53} - 1$, Protobuf and REST payloads serialize `url_hash` strictly as decimal strings.

4.2 Query Normalization, FTS5 Grouping & Multi-Signal Ranking

Literal full-text matching struggles on conversational queries (for example, *"what is the main purpose of the joining in SQL"* or *"when bangladesh is established"*). Figure 3 outlines the eight-stage transformation pipeline implemented across `spaces/crawler/db_turso.py` and `spaces/crawler/synthesis.py`.

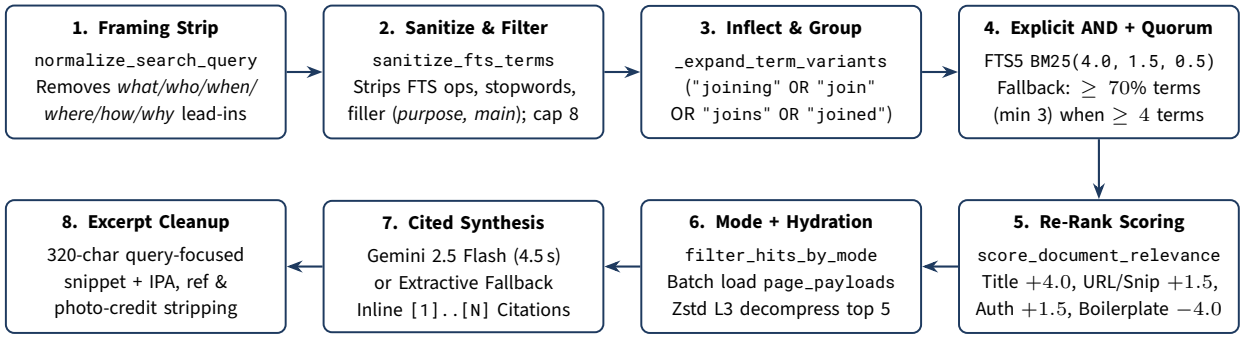


Figure 3: Low-Level Design: Query sanitization, FTS5 grouping, re-ranking, and synthesis pipeline.

In stage 3, parenthesized inflection groups are joined with explicit `AND` rather than implicit whitespace to satisfy SQLite FTS5 parser rules. `_expand_term_variants` covers regular `-ing`, `-ies`, `-ed`, sibilant `-es` (`indexes` → `index`,

`classes` → `class`), and plural `-s` forms. Stage 5 sorts candidates by (`matched_terms, rel_score, db_score`) using host-boundary validation (`_is_authoritative_host`) and penalizes generic repository template phrases when `mode != "code"`.

4.3 Live Discovery, Parsing & Bounded Deduplication

1. **Two-Pass Domain Diversity (`discovery.py`):** Concurrently queries DuckDuckGo HTML, Wikipedia API, HackerNews Algolia, Reddit JSON, arXiv Atom, GitHub Search (plus optional Brave Search) excluding domains in the 24 h WAF cooldown cache. Pass 1 selects at most 1 URL per domain; Pass 2 backfills remaining slots up to `max_urls` from multi-article hubs (`en.wikipedia.org`, `arxiv.org`, `github.com`, `stackoverflow.com`).
2. **Polite Chrome-Impersonated Fetch (`robots.py, parser.py`):** Fetches `robots.txt` per domain under an `asyncio.Lock` (24 h TTL, supporting fractional `Crawl-delay`), then executes `curl_cffi` (`impersonate="chrome124"`) capped at 12 concurrent fetches. `selectolax.lexbor` extracts `__NEXT_DATA__` SSR JSON, Schema.org JSON-LD metadata, structured Markdown, and up to `max_links=80` followable outbound links while filtering Cloudflare/Akamai challenge pages (`is_waf_or_empty_page`).
3. **Deduplication & Compression (`bloom.py, dedup.py, compression.py`):** A 10^6-capacity `URLBloomFilter` (`mmh3` + `bitarray`, `0.1%` FP rate) warms from `search_index` at boot. Before saving, `compute_simhash` hashes unigrams and 2-gram shingles over the leading `12,000` characters (`Hamming <= 3` per domain), avoiding multi-second CPU stalls on 2 MB encyclopedia pages, and `zstd.ZstdCompressor(level=3)` compresses full Markdown into `page_payloads`.

5. Storage Schema (`Turso LibSQL FTS5`)

All tables, FTS5 virtual indexes, synchronization triggers, compressed blobs, and crawl queues reside in one serverless Turso LibSQL database (`migrations/03_turso_fts5_schema.sql`), illustrated in Figure 4.

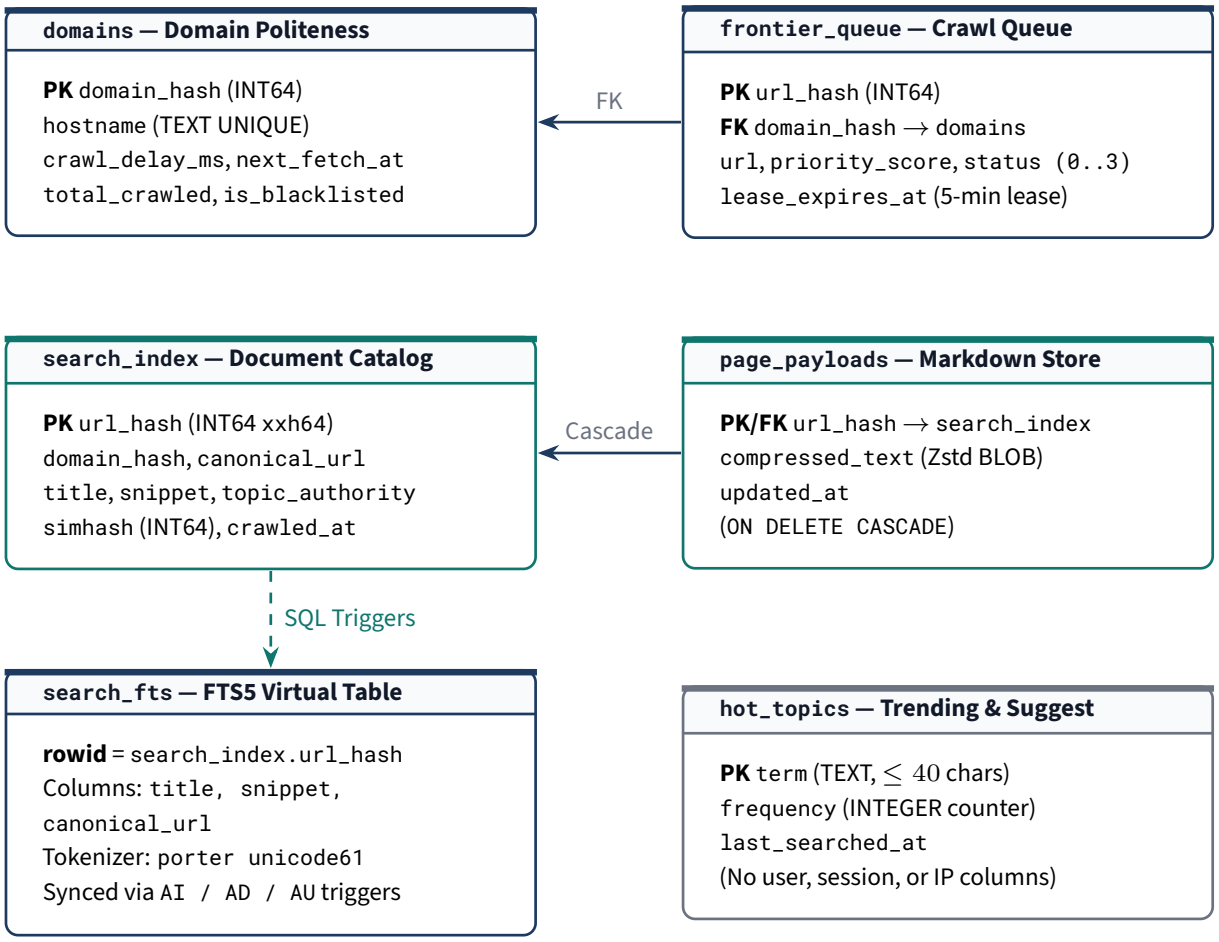


Figure 4: Turso LibSQL relational and FTS5 schema (`migrations/03_turso_fts5_schema.sql`).

6. Latency Budgets, Security Controls & Tooling

Figure 5 summarizes the configured stage budgets defined in `services/common/config.py`. Tight per-fetch (3.5 s), Gemini (4.5 s), and JIT batch (8.5 s) ceilings keep streaming responses interactive while still completing Turso writes on free-tier compute.

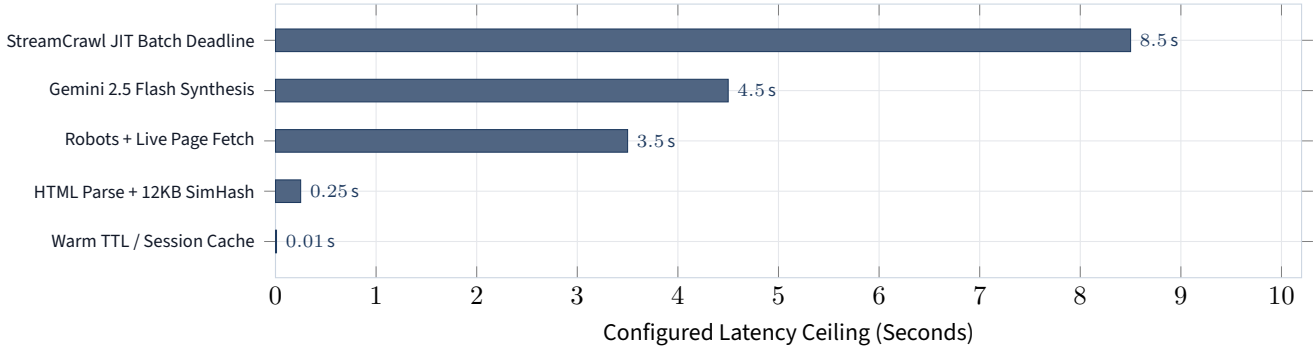


Figure 5: Configured latency envelopes across cache, crawl, and synthesis tiers.

Workflow / Utility	Function
`.github/workflows/ci_cd.yml`	Runs `compileall`, `ruff check --select E9,F63,F7,F82,F401`, Vite frontend build (`tsc -b && vite build`), 44 unit/integration tests, and Turso schema migration.
`.github/workflows/deploy_pages.yml`	Builds `frontend/` targeting `VITE_API_BASE_URL` (`https://neutron-gateway.onrender.com`) and publishes static bundles and PDF docs to GitHub Pages.
`scripts/gen_proto.py`	Regenerates Python ConnectRPC and Protobuf stubs from `proto/neutron/v1/neutron.proto`.
`scripts/refresh_stale.py`	Queries oldest `search_index` rows (`row.asdict()`), verifies `robots.txt` and `text/html`, and updates Markdown payloads with full `1,200`-char snippets.

Table 3: Continuous integration workflows and maintenance utilities.

Privacy & Frontend Security Summary

- Rate limiting (`30/min /search`, `15/min /search/stream`, `20/min /api/research`) uses volatile in-memory `X-Forwarded-For` buckets and sets `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`, and `Permissions-Policy: interest-cohort=()`.
- Search snippets render strictly via React text nodes and `<mark>` spans (no `dangerouslySetInnerHTML`); Reader Markdown passes through `marked` + `DOMPurify` (`ALLOWED_URI_REGEXP: ^https?://`); and outbound anchors are filtered with `safeHttpUrl`.